

Object-Oriented Programming (OOP)



Some early object-oriented systems only had objects and messages, and did not yet have the concept of classes.

Currently, object-oriented programming is defined based on three core elements:

- class
- inheritance
- dynamic binding.

Some features of OOP can be implemented in C or procedural languages, but the difference lies in the fact that OOP languages explicitly support and require the use of these concepts within the language.

The level of OOP support varies between languages.

Eiffel is considered fully object-oriented, Java is very highly object-oriented, and C++ only supports it partially (a combination of object-oriented and procedural approaches).

OOP is a programming methodology:

- Accurately describe objects in the world;
- Use objects as the foundation for building algorithms;

- The design revolves around the system's data;
- The program is divided into object classes;
- The data is packaged, concealed, and protected;
- The parties communicate with each other through notifications;
- The program is designed using a bottom-up approach.

Object-Oriented Systems

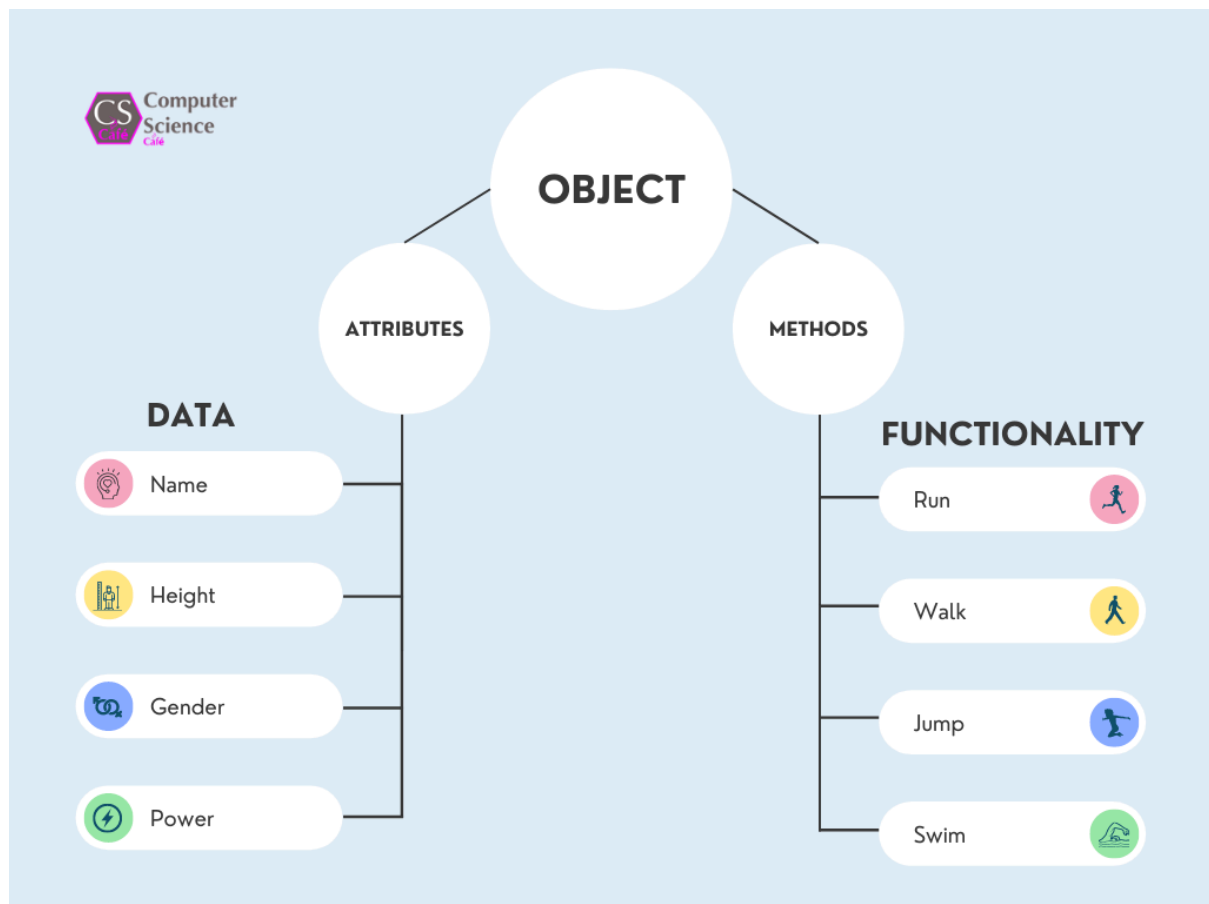
An object-oriented system consists of a set of **object**.

Each object is a **pack** of the two components:

- data (object properties) and operations that process that data.

Potential subjects **inheritance** The properties are derived from other objects, allowing for reuse and expansion.

The system operates based on **interaction between objects** This is done through a message communication mechanism. This process involves sending and receiving messages between entities.



Object

In programming, a program can be viewed as a complex "machine," created from many components such as functions and structures. However, to better model the real world, people use...**object** as a new “material”.

Writing object-oriented programs means building models of real-world components through objects.

Characteristics of the object

The object has characteristics **pack** That is, combining data and processing within the same entity.

Attributes are data belonging to an object.

Methods are operations that process that data.

Object structure

An object functions like a "black box," containing data inside and methods for interaction outside.

Methods are generally divided into four groups:

- object creation and destruction group,
- information retrieval group,
- business process team,
- operator group.

Three elements define an object.

An object is defined by:

- Identification: helps to distinguish objects from one another.
- State: is the current value of the attributes.
- Activities: These are the actions that an object can perform.

Components of the object

An object consists of:

- identification
- attribute (data)

- behavior (method).

Each object is an independent entity, always having its own identifier. In programming languages like C++, this identifier can be an address (handle). Two objects may have the same data but are still two different entities because their identifiers are different.

Class

An object is a concrete entity that exists within a system, while a class is an abstract concept used to describe a collection of objects of the same type.

For example, "car" is the class, and each specific car is an object.

The class used for modeling:

- Attributes (data) of an object
- The object's method (behavior)

Attributes are characteristics of an object, which can be constants, variables, or user-defined types.

Methods are functions that process data, have a return type, and are members of a class.

Message and message delivery

A message is the way one object requests another object to perform an action.

A message includes the recipient, the method name, and the parameters.

Communicating a message is the process of:

The requesting party submits the request, verifies its validity, and executes the corresponding method.

Encapsulation

Encapsulation is a mechanism that combines data and methods within a single object, while concealing the details of its internal implementation.

Users interact only through the interface (methods), without needing to know how things work internally.

Benefit:

- Data protection
- Easy to manage change

- Reduce errors and make fewer code edits.

Inheritance

Inheritance allows the creation of new classes from existing classes, inheriting their attributes and methods.

Subclasses can:

- Reuse
- Expand or overwrite

Meaning:

- Reuse source code
- Avoid code repetition.
- Easy to maintain

There are two types:

- Inheritance rules: a child class can have only one parent class.
- Multiple inheritance: a child class can have multiple parent classes (some languages like Java do not support this).

Polymorphism

Polymorphism is the ability of a method to have the same name but behave differently depending on the object.

Same method call, but:

- Different object → different processing → different result

Performed by:

- Overloading
- Overriding

Meaning:

- Flexibility in programming
- Increase system scalability.